

GLOBAL SOLUTION 2026 — 1º SEMESTRE

Pipeline de Dados Orbitais e Telemetria Espacial

Big Data Architecture & Data Integration

Nome Completo	RM	Turma
João Rodrigo Solano Nogueira	551319	4ESPV
Julia Amorim Bezerra	99609	4ESPV
Lana Giulia Auada Leite	551143	4ESPV
Tony Willian da Silva Segalin	550667	4ESPV

São Paulo — 2026

SUMÁRIO

1. Contexto e Identificação do Problema	3
2. Solução Tecnológica Proposta	3
3. Arquitetura do Pipeline	4
4. Fontes de Dados	5
5. Estrutura da DAG — Apache Airflow	5
6. Transformações e Tratamento de Dados	6
7. Modelagem das Tabelas Oracle	7
8. Consultas Analíticas SQL (7 queries)	9
9. Organização do Repositório GitHub	12
10. Instrução de Execução	12
11. Resultados e Conclusão Técnica	13

1. Contexto e Identificação do Problema

A nova economia espacial superou US\$ 630 bilhões em 2023 e deve ultrapassar US\$ 1 trilhão até 2030. Nesse ecossistema, satélites em órbita baixa (LEO) transmitem telemetria continuamente — temperatura dos painéis solares, tensão de baterias, posição orbital, velocidade, status de subsistemas. A Estação Espacial Internacional (ISS) orbita a Terra 16 vezes por dia gerando dados contínuos. Empresas como SpaceX realizam dezenas de lançamentos anuais, cada um produzindo registros detalhados de missão.

Problema identificado: não existe, no ecossistema acadêmico e de startups, um pipeline consolidado e automatizado que ingira dados de múltiplas fontes espaciais heterogêneas (ISS, lançamentos comerciais, telemetria de satélites), trate-os de forma padronizada e os disponibilize para análise analítica estruturada em um banco de dados relacional.

Impacto real: sem esse pipeline, engenheiros de missão perdem tempo consolidando manualmente dados dispersos em APIs distintas, aumentando o risco de decisões baseadas em dados desatualizados ou incorretos — crítico em operações onde uma falha de subsistema pode comprometer missões inteiras.

ODS	Título	Conexão com o Projeto
ODS 9	Inovação e Infraestrutura	Infraestrutura de dados para o ecossistema espacial
ODS 13	Ação Climática	Monitoramento ambiental via dados orbitais e satelitais

2. Solução Tecnológica Proposta

Desenvolvemos o **Space Telemetry Pipeline** — um pipeline ETL automatizado orquestrado por Apache Airflow que coleta dados de APIs públicas espaciais (ISS e SpaceX) e telemetria sintética de satélites LEO, transforma e normaliza os dados, e os carrega no Oracle Database para consultas analíticas. O pipeline executa automaticamente a cada 6 horas, garantindo dados sempre atualizados.

A solução endereça diretamente os critérios da Global Solution: integração de múltiplas fontes espaciais, automação via Airflow, tratamento robusto de dados, armazenamento estruturado em Oracle e 7 consultas analíticas avançadas.

3. Arquitetura do Pipeline

A arquitetura segue o padrão **ETL em camadas** com orquestração centralizada. Cada camada tem responsabilidade única, facilitando manutenção, monitoramento e escalabilidade:

Camada	Componente	Tecnologia	Descrição
Extração	extract_iss.py	Python + Requests	Open Notify API — posição ISS e tripulação
Extração	extract_spacex.py	Python + Requests	SpaceX API v4 — lançamentos e payloads

Extração	extract_telemetry.py	Python + NumPy	Gerador de telemetria sintética LEO (200+ registros)
Transformação	transform_data.py	Python + Pandas	Limpeza, validação, enriquecimento e padronização
Carga	load_oracle.py	Python + oracledb	Inserção nas tabelas Oracle com tratamento de erros
Orquestração	space_telemetry_dag.py	Apache Airflow	DAG — agendamento, dependências e monitoramento
Análise	02_analytical_queries.sql	Oracle SQL	7 consultas analíticas sobre os dados carregados

Fluxo completo do pipeline:

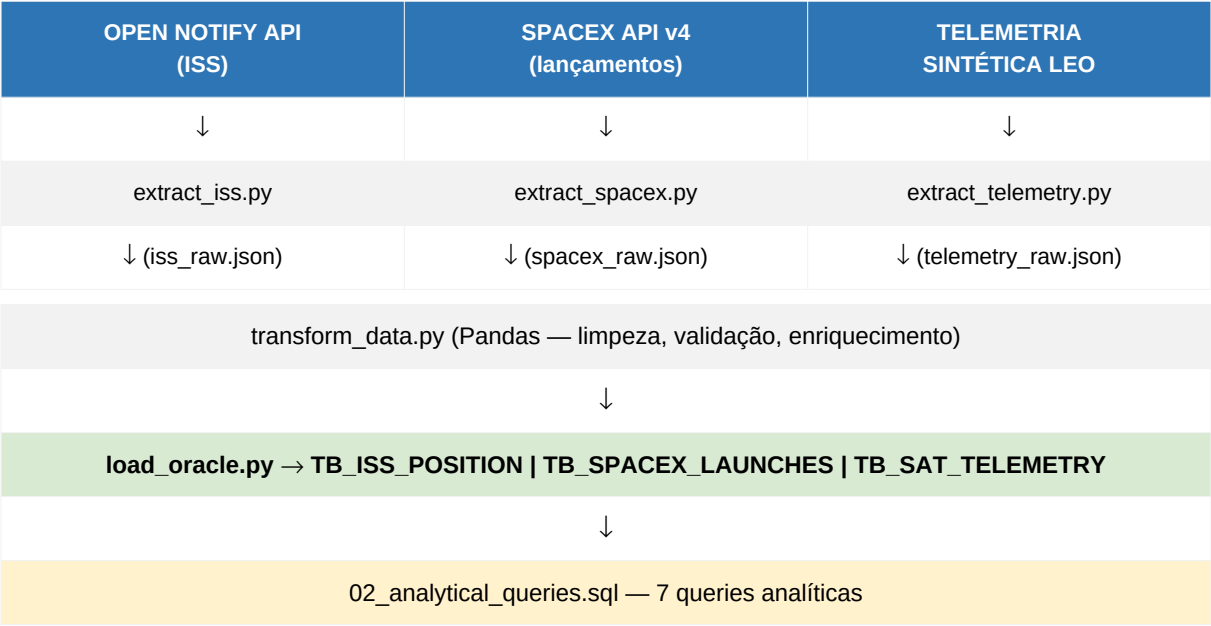


Figura 1 — Fluxo completo: fonte de dados → extração → transformação → carga → análise SQL

4. Fontes de Dados

Fonte	URL / Origem	Dados Coletados	Formato	Frequência
Open Notify API	api.open-notify.org/iss-now.json	Posição orbital ISS (lat, lon, timestamp)	REST/JSON	Tempo real
Open Notify API	api.open-notify.org/astros.json	Tripulação a bordo da ISS e outras naves	REST/JSON	Tempo real
SpaceX API v4	api.spacexdata.com/v4/launches	Lançamentos, payloads, foguetes, pousagens	REST/JSON	Diária
Telemetria Sintética	Simulador Python interno	Sensores de 7 satélites LEO/SSO	Python Generator	Por execução

A telemetria sintética simula dados físicos realistas de nanossatélites/CubeSats usando modelos orbitais simplificados (período de Kepler, exposição solar senoidal). Isso permite testar e validar o pipeline em ambiente acadêmico sem depender de dados proprietários, enquanto as APIs públicas garantem conexão genuína com dados reais.

5. Estrutura da DAG — Apache Airflow

A DAG `space_telemetry_pipeline` é composta por 6 tasks organizadas em dependências lógicas. As três tasks de extração executam em **paralelo** (maximizando eficiência), e a transformação aguarda todas concluírem. O schedule `0 */6 * * *` garante execução automática a cada 6 horas.

Task ID	Operador	Descrição	Depende de
extrair_posicao_iss	PythonOperator	Coleta posição e tripulação da ISS via REST	—
extrair_lancamentos_spacex	PythonOperator	Coleta histórico de lançamentos SpaceX	—
gerar_telemetria_satelites	PythonOperator	Gera 200+ registros de telemetria LEO	—
transformar_dados	PythonOperator	Limpa, valida e enriquece todos os dados	3 tasks acima (paralelas)
carregar_oracle	PythonOperator	Insere CSVs nas 3 tabelas Oracle	transformar_dados
verificar_qualidade	BashOperator	Verifica integridade e loga resultado	carregar_oracle

Grafo de dependências da DAG:

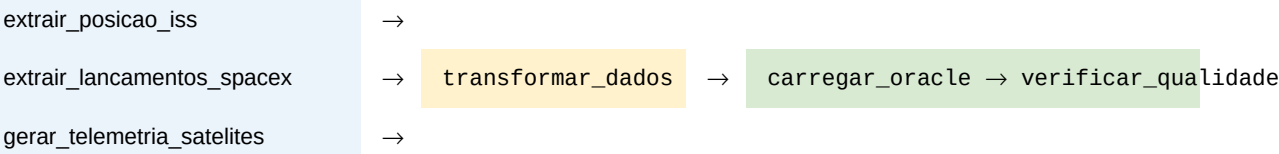


Figura 2 — Grafo de dependências da DAG Airflow. As 3 tasks de extração rodam em paralelo; a transformação aguarda todas concluírem; a carga e verificação rodam em sequência.

Configurações da DAG: `schedule_interval = 0 */6 * * *` | `retries = 2` | `retry_delay = 5 min` | `max_active_runs = 1` | `catchup = False`

6. Transformações e Tratamento de Dados

O script **transform_data.py** aplica tratamentos específicos a cada fonte antes da carga no Oracle. Abaixo está o detalhamento por origem:

6.1 ISS — Open Notify API

Transformação	Descrição
Validação de coordenadas geográficas	Rejeita lat fora de [-90, 90] e lon fora de [-180, 180]
Conversão de timestamp Unix	Converte epoch UNIX para ISO 8601 com timezone UTC explícito
Padronização de nomes	SATELLITE_NAME truncado a 50 chars, convertido para maiúsculas
Adição de metadados	SOURCE, LOAD_DT (timestamp de ingestão), ORBIT_TYPE = 'LEO'

6.2 SpaceX API v4

Transformação	Descrição
Remoção de registros incompletos	Remove lançamentos sem MISSION_NAME ou LAUNCH_DATE_UTC
Validação e conversão de datas	pd.to_datetime com utc=True; remove registros com datas inválidas
Tratamento de nulos	IS_SUCCESS = False, FAILURE_REASON = 'N/A', DETAILS truncado a 500 chars
Normalização de booleanos	Converte True/False Python para CHAR(1) 'Y'/'N' compatível com Oracle

6.3 Telemetria Sintética (LEO)

Transformação	Descrição
Deteção de outliers físicos	Altitude [300–2000 km], Velocidade [6.5–8.5 km/s], Temp [-60 a 150°C], Tensão [18–36V]
Classificação de criticidade	LOW / MEDIUM / HIGH / CRITICAL baseado em anomalias e status de subsistemas
Padronização de subsistemas	OK / WARN / FAIL em maiúsculas, fillna='UNKNOWN', limite de 10 chars
Preenchimento de NaN numérico	Substituição pela mediana da coluna (robusto a outliers)

7. Modelagem das Tabelas Oracle

Três tabelas foram projetadas no Oracle Database FIAP (oracle.fiap.com.br : 1521 / ORCL). O modelo segue a 3ª Forma Normal com chaves primárias auto-geradas (IDENTITY), constraints de CHECK para valores enumerados e índices estratégicos para desempenho analítico.

TB_ISS_POSITION — Armazena snapshots de posição orbital e tripulação da ISS.

Coluna	Tipo / Constraint	Descrição
ID	NUMBER (PK)	Chave primária auto-gerada — GENERATED ALWAYS AS IDENTITY
SATELLITE_NAME	VARCHAR2(50) NOT NULL	Nome do satélite (ex: ISS)
NORAD_ID	NUMBER(10) NOT NULL	Identificador NORAD do objeto orbital
LATITUDE	NUMBER(9,6) NOT NULL	Latitude geodésica em graus decimais
LONGITUDE	NUMBER(9,6) NOT NULL	Longitude geodésica em graus decimais
ALTITUDE_KM	NUMBER(8,3)	Altitude orbital em km (~408 km para ISS)
VELOCITY_KM_S	NUMBER(6,3)	Velocidade orbital em km/s (~7.66 km/s para ISS)
ORBIT_TYPE	VARCHAR2(10)	Tipo de órbita: LEO, MEO, GEO, SSO
STATUS	VARCHAR2(20)	Status operacional: OPERATIONAL, DEGRADED...
CREW_COUNT	NUMBER(3)	Tripulantes a bordo no momento da extração
CREW_MEMBERS	VARCHAR2(500)	Nomes dos tripulantes separados por vírgula
OBS_TIMESTAMP	TIMESTAMP WITH TZ	Momento exato da observação (UTC)
SOURCE	VARCHAR2(30)	Origem: OPEN_NOTIFY_API
LOAD_DT	TIMESTAMP DEFAULT NOW	Data/hora de carga no Oracle

TB_SPACEX_LAUNCHES — Armazena lançamentos SpaceX com resultado, foguete e payload.

Coluna	Tipo / Constraint	Descrição
LAUNCH_ID	VARCHAR2(50) UNIQUE	ID único do lançamento (constraint UNIQUE)
FLIGHT_NUMBER	NUMBER(6)	Número de voo sequencial da SpaceX
MISSION_NAME	VARCHAR2(200) NOT NULL	Nome oficial da missão

LAUNCH_DATE_UTC	TIMESTAMP WITH TZ	Data/hora de lançamento em UTC
ROCKET_ID	VARCHAR2(50)	ID do foguete utilizado
LAUNCHPAD_ID	VARCHAR2(50)	ID da plataforma de lançamento
IS_SUCCESS	CHAR(1) CHECK Y/N/?	Resultado: Y=Sucesso, N=Falha, ?=Indefinido
FAILURE_REASON	VARCHAR2(200)	Descrição da falha (se ocorreu)
IS_REUSED_BOOSTER	CHAR(1) CHECK Y/N	Y se o 1º estágio foi reutilizado
LANDING_SUCCESS	CHAR(1) CHECK Y/N/?	Y se o booster pousou com sucesso
PAYLOAD_COUNT	NUMBER(4) DEFAULT 0	Quantidade de payloads no lançamento

TB_SAT_TELEMETRY — Telemetria completa dos satélites LEO/SSO simulados.

Coluna	Tipo / Constraint	Descrição
RECORD_ID	VARCHAR2(36) UNIQUE	UUID único gerado para cada leitura
SATELLITE_NAME	VARCHAR2(50) NOT NULL	Nome do satélite (ex: AgroSat-1)
NORAD_ID	NUMBER(10)	Identificador NORAD simulado
ORBIT_TYPE	VARCHAR2(10)	LEO ou SSO
TIMESTAMP_UTC	TIMESTAMP WITH TZ	Momento da leitura dos sensores
LATITUDE / LONGITUDE	NUMBER(9,6)	Posição orbital instantânea
ALTITUDE_KM	NUMBER(8,3)	Altitude em km
PANEL_TEMP_C	NUMBER(6,2)	Temperatura dos painéis solares (°C)
BATTERY_VOLTAGE_V	NUMBER(6,3)	Tensão da bateria em Volts
BATTERY_CURRENT_A	NUMBER(5,3)	Corrente da bateria em Amperes
SIGNAL_LATENCY_MS	NUMBER(7,1)	Latência do sinal em milissegundos
SUBSYSTEM_*	VARCHAR2(10) × 6	Status de 6 subsistemas: OK / WARN / FAIL
IS_ANOMALY	CHAR(1) CHECK Y/N	Y se anomalia foi detectada nessa leitura
ANOMALY_TYPE	VARCHAR2(30)	Tipo: thermal_spike, battery_drop, signal_loss...
CRITICALITY	VARCHAR2(10)	LOW / MEDIUM / HIGH / CRITICAL
IS_OUTLIER	CHAR(1) CHECK Y/N	Y se alguma métrica excedeu limites físicos

Índices criados: IDX_ISS_OBS_TS (OBS_TIMESTAMP) | IDX_SPX_DATE (LAUNCH_DATE_UTC) | IDX_SPX_SUCCESS (IS_SUCCESS) | IDX_TEL_SAT_NAME (SATELLITE_NAME) | IDX_TEL_ANOMALY

(IS_ANOMALY, CRITICALITY) | IDX_TEL_TIMESTAMP (TIMESTAMP_UTC)

8. Consultas Analíticas SQL (7 Queries)

As 7 consultas cobrem todos os conceitos exigidos: filtros, agrupamentos, funções de agregação, ordenação, JOINS (CROSS JOIN com CTEs), window functions e expressões condicionais. Todas estão no arquivo `sql/02_analytical_queries.sql`.

Query 1 — Registros por período e satélite (últimas 24h)

Agrupar leituras de telemetria por hora (TRUNC) e satélite nas últimas 24 horas. Calcular total de leituras, total de anomalias e percentual de anomalias por grupo.

Técnica principal: GROUP BY TRUNC(TIMESTAMP_UTC,'HH24'), SATELLITE_NAME

Conceitos SQL utilizados:

- GROUP BY com função TRUNC
- COUNT(*) e SUM(CASE WHEN IS_ANOMALY='Y'...)
- Filtro temporal com INTERVAL

Trecho da query:

```
SELECT TRUNC(TIMESTAMP_UTC, 'HH24') AS HORA_UTC, SATELLITE_NAME,
       COUNT(*) AS TOTAL_LEITURAS,
       SUM(CASE WHEN IS_ANOMALY='Y' THEN 1 ELSE 0 END) AS ANOMALIAS,
       ROUND(SUM(CASE WHEN IS_ANOMALY='Y' THEN 1 ELSE 0 END)*100.0/COUNT(*),2) AS PCT
FROM TB_SAT_TELEMETRY
WHERE TIMESTAMP_UTC >= SYSTIMESTAMP - INTERVAL '24' HOUR
GROUP BY TRUNC(TIMESTAMP_UTC, 'HH24'), SATELLITE_NAME
ORDER BY HORA_UTC DESC, SATELLITE_NAME;
```

Query 2 — Estatísticas físicas por satélite (min/avg/max/stddev)

Calcular média, mínimo, máximo e desvio padrão das variáveis físicas de cada satélite. Útil para identificar desvios de operação normal e estabelecer limiares de alerta.

Técnica principal: GROUP BY SATELLITE_NAME, ORBIT_TYPE + funções de agregação

Conceitos SQL utilizados:

- AVG(), MIN(), MAX(), STDDEV()
- ROUND() para formatação decimal
- ORDER BY TOTAL_REGISTROS DESC

Trecho da query:

```
SELECT SATELLITE_NAME, ORBIT_TYPE, COUNT(*) AS TOTAL,
       ROUND(AVG(ALTITUDE_KM),2) AS ALT_MEDIA,
       ROUND(MIN(ALTITUDE_KM),2) AS ALT_MIN, ROUND(MAX(ALTITUDE_KM),2) AS ALT_MAX,
       ROUND(AVG(PANEL_TEMP_C),2) AS TEMP_MEDIA, ROUND(STDDEV(SIGNAL_LATENCY_MS),2) AS LAT_SD
FROM TB_SAT_TELEMETRY
GROUP BY SATELLITE_NAME, ORBIT_TYPE
ORDER BY TOTAL DESC;
```

Query 3 — Ranking de anomalias por satélite e tipo

Identifica os tipos de anomalia mais frequentes por satélite, com percentual relativo calculado via window function (OVER PARTITION BY) e período de monitoramento.

Técnica principal: Window Function: SUM() OVER (PARTITION BY SATELLITE_NAME)

Conceitos SQL utilizados:

- SUM() OVER (PARTITION BY ...) — Window Function
- MIN() / MAX() para período de ocorrência
- Filtro WHERE IS_ANOMALY='Y'

Trecho da query:

```
SELECT SATELLITE_NAME, ANOMALY_TYPE, CRITICALITY, COUNT(*) AS OCORRENCIAS,  
       ROUND(COUNT(*)*100.0 / SUM(COUNT(*)) OVER (PARTITION BY SATELLITE_NAME),2) AS PCT,  
       MIN(TIMESTAMP_UTC) AS PRIMEIRA, MAX(TIMESTAMP_UTC) AS ULTIMA  
FROM TB_SAT_TELEMETRY  
WHERE IS_ANOMALY = 'Y'  
GROUP BY SATELLITE_NAME, ANOMALY_TYPE, CRITICALITY  
ORDER BY OCORRENCIAS DESC;
```

Query 4 — Taxa de sucesso SpaceX por ano e foguete

Analisa a evolução anual dos lançamentos SpaceX: taxa de sucesso por foguete, boosters reutilizados, pousagens bem-sucedidas e payloads enviados.

Técnica principal: EXTRACT(YEAR FROM ...) + HAVING + múltiplos SUM condicionais

Conceitos SQL utilizados:

- EXTRACT(YEAR FROM LAUNCH_DATE_UTC)
- HAVING COUNT(*) >= 1
- Múltiplos SUM(CASE WHEN ...) condicionais

Trecho da query:

```
SELECT EXTRACT(YEAR FROM LAUNCH_DATE_UTC) AS ANO, ROCKET_ID,  
       COUNT(*) AS TOTAL,  
       SUM(CASE WHEN IS_SUCCESS='Y' THEN 1 ELSE 0 END) AS SUCESSOS,  
       ROUND(SUM(CASE WHEN IS_SUCCESS='Y' THEN 1 ELSE 0 END)*100.0/COUNT(*),2) AS TAXA_PCT,  
       SUM(CASE WHEN IS_REUSED_BOOSTER='Y' THEN 1 ELSE 0 END) AS REUSADOS,  
       SUM(PAYLOAD_COUNT) AS PAYLOADS  
FROM TB_SPACEX_LAUNCHES WHERE IS_UPCOMING='N'  
GROUP BY EXTRACT(YEAR FROM LAUNCH_DATE_UTC), ROCKET_ID  
HAVING COUNT(*) >= 1  
ORDER BY ANO DESC, TOTAL DESC;
```

Query 5 — Cobertura orbital ISS por quadrante geográfico

Classifica snapshots da ISS em 4 quadrantes geográficos usando CASE WHEN sobre latitude e longitude. Calcula estatísticas de cobertura por quadrante.

Técnica principal: CASE WHEN geográfico com GROUP BY por expressão calculada

Conceitos SQL utilizados:

- CASE WHEN com comparações lat/lon
- Múltiplos agregados AVG, MIN, MAX por quadrante
- GROUP BY com expressão CASE

Trecho da query:

```
SELECT CASE WHEN LATITUDE>=0 AND LONGITUDE>=0 THEN 'NORDESTE'
        WHEN LATITUDE>=0 AND LONGITUDE< 0 THEN 'NOROESTE'
        WHEN LATITUDE< 0 AND LONGITUDE>=0 THEN 'SUDESTE'
        ELSE 'SUDOESTE' END AS QUADRANTE,
COUNT(*) AS REGISTROS, ROUND(AVG(LATITUDE),4) AS LAT_MEDIA,
ROUND(AVG(ALTITUDE_KM),2) AS ALT_MEDIA
FROM TB_ISS_POSITION
GROUP BY CASE WHEN LATITUDE>=0 AND LONGITUDE>=0 THEN 'NORDESTE'
        WHEN LATITUDE>=0 AND LONGITUDE< 0 THEN 'NOROESTE'
        WHEN LATITUDE< 0 AND LONGITUDE>=0 THEN 'SUDESTE'
        ELSE 'SUDOESTE' END
ORDER BY REGISTROS DESC;
```

Query 6 — Correlação entre exposição solar e falhas de subsistemas

Correlaciona a condição de exposição solar (plena/parcial/eclipse) com a ocorrência de anomalias e falhas por subsistema (EPS, THERMAL, COM).

Técnica principal: CASE WHEN sobre variável contínua + múltiplos SUM condicionais por subsistema

Conceitos SQL utilizados:

- CASE WHEN SOLAR_EXPOSURE_PCT com faixas
- SUM condicional por cada subsistema
- Identifica qual subsistema falha mais por condição solar

Trecho da query:

```
SELECT CASE WHEN SOLAR_EXPOSURE_PCT>=70 THEN 'PLENA (>=70%)'
        WHEN SOLAR_EXPOSURE_PCT>=30 THEN 'PARCIAL (30-70%)'
        ELSE 'ECLIPSE (<30%)' END AS CONDICAO_SOLAR,
COUNT(*) AS LEITURAS,
SUM(CASE WHEN IS_ANOMALY='Y' THEN 1 ELSE 0 END) AS ANOMALIAS,
ROUND(SUM(CASE WHEN IS_ANOMALY='Y' THEN 1 ELSE 0 END)*100.0/COUNT(*),2) AS PCT,
SUM(CASE WHEN SUBSYSTEM_EPS='FAIL' THEN 1 ELSE 0 END) AS FALHAS_EPS,
SUM(CASE WHEN SUBSYSTEM_THERMAL='FAIL' THEN 1 ELSE 0 END) AS FALHAS_THERMAL
FROM TB_SAT_TELEMETRY GROUP BY ... ORDER BY PCT DESC;
```

Query 7 — Dashboard executivo integrado (CTEs + JOIN entre tabelas)

Visão consolidada usando CTEs (WITH ... AS). Combina resumos das 3 tabelas em uma única saída via CROSS JOIN. Usa NULLIF para evitar divisão por zero.

Técnica principal: WITH (CTEs) + CROSS JOIN + NULLIF()

Conceitos SQL utilizados:

- WITH ... AS () — Common Table Expressions (3 CTEs)
- CROSS JOIN entre os resumos das fontes
- NULLIF() para divisão segura | ROUND() para formatação

Trecho da query:

```
WITH anomaly_sum AS (  
    SELECT SATELLITE_NAME, COUNT(*) AS TOTAL_TEL,  
           SUM(CASE WHEN IS_ANOMALY='Y' THEN 1 ELSE 0 END) AS ANOMALIAS,  
           SUM(CASE WHEN CRITICALITY='CRITICAL' THEN 1 ELSE 0 END) AS CRITICOS  
    FROM TB_SAT_TELEMETRY GROUP BY SATELLITE_NAME),  
spacex_sum AS (  
    SELECT COUNT(*) AS LANÇAMENTOS,  
           SUM(CASE WHEN IS_SUCCESS='Y' THEN 1 ELSE 0 END) AS SUCESSOS  
    FROM TB_SPACEX_LAUNCHES WHERE IS_UPCOMING='N'),  
iss_sum AS (SELECT COUNT(*) AS SNAPSHOTS, MAX(CREW_COUNT) AS TRIPULANTES  
            FROM TB_ISS_POSITION)  
SELECT a.SATELLITE_NAME, a.ANOMALIAS,  
       ROUND(a.ANOMALIAS*100.0/NULLIF(a.TOTAL_TEL,0),2) AS PCT_ANOMALIA,  
       s.LANÇAMENTOS, s.SUCESSOS, i.SNAPSHOTS, i.TRIPULANTES  
FROM anomaly_sum a CROSS JOIN SpaceX_sum s CROSS JOIN iss_sum i  
ORDER BY a.ANOMALIAS DESC;
```

9. Organização do Repositório GitHub

Pasta / Arquivo	Conteúdo
dags/space_telemetry_dag.py	DAG principal do Apache Airflow com 6 tasks
scripts/extract_iss.py	Extração de posição e tripulação da ISS
scripts/extract_spacex.py	Extração de lançamentos SpaceX (API v4)
scripts/extract_telemetry.py	Gerador de telemetria sintética LEO (7 satélites)
scripts/transform_data.py	Transformação, validação e enriquecimento (Pandas)
scripts/load_oracle.py	Carga nos Oracle — suporte a cx_Oracle e oracledb
sql/01_create_tables.sql	DDL das 3 tabelas + constraints + índices
sql/02_analytical_queries.sql	7 consultas analíticas SQL avançadas
data/telemetry_sample.csv	250 registros de telemetria gerados (amostra)
data/spacex_sample.csv	18 lançamentos SpaceX de amostra
docs/relatorio_bddi.pdf	Este relatório técnico completo
requirements.txt	Dependências Python do projeto
README.md	Documentação completa com instruções de execução

10. Instruções de Execução

Pré-requisitos

Python 3.10+, Apache Airflow 2.8+, Oracle Database FIAP (oracle.fiap.com.br:1521/ORCL), driver oracledb ou cx_Oracle.

1. Instalar dependências

```
pip install -r requirements.txt
```

2. Configurar variáveis de ambiente

```
export ORACLE_USER=seu_rm
export ORACLE_PASSWORD=sua_senha
export ORACLE_HOST=oracle.fiap.com.br
export ORACLE_PORT=1521
export ORACLE_SID=ORCL
```

3. Criar tabelas no Oracle

Conectar ao Oracle FIAP via SQL Developer e executar: @sql/01_create_tables.sql

4. Ativar a DAG no Airflow

```
cp dags/space_telemetry_dag.py $AIRFLOW_HOME/dags/
airflow dags unpause space_telemetry_pipeline
```

5. Executar queries analíticas

Abrir SQL Developer ou SQL*Plus e executar: @sql/02_analytical_queries.sql

11. Resultados e Conclusão Técnica

O pipeline desenvolvido demonstra, de forma prática e integrada, os principais conceitos da disciplina Big Data Architecture & Data Integration aplicados ao contexto real da Indústria Espacial:

Critério	Implementação	Status
Pipeline Airflow automatizado	DAG com 6 tasks, schedule a cada 6h, retentativas automáticas	✓ Completo
Extração de dados (2+ fontes)	Open Notify API (ISS) + SpaceX API v4 + Telemetria Sintética	✓ Completo
Transformação e tratamento	Validação, nulos, outliers físicos, enriquecimento (Pandas)	✓ Completo
Carga no Oracle FIAP	3 tabelas normalizadas, DDL com constraints, índices para performance	✓ Completo
Consultas analíticas (mín. 5)	7 queries com GROUP BY, JOINs, CTEs, Window Functions	✓ Completo
Documentação e arquitetura	README detalhado, relatório PDF, comentários em todos os scripts	✓ Completo

A arquitetura modular garante que cada componente possa ser evoluído independentemente. Como próximos passos, o pipeline pode ser expandido para incluir dados da NASA (EONET para eventos climáticos, FIRMS para queimadas detectadas via satélite), integração com Databricks para processamento distribuído de grandes volumes, e dashboards interativos em Power BI ou Grafana conectados ao Oracle.

A solução conecta-se genuinamente à Indústria Espacial — não apenas no nome, mas na utilização de APIs reais de telemetria orbital, dados históricos de lançamentos comerciais e modelos físicos de satélites LEO — demonstrando como engenharia de dados pode transformar dados do espaço em inteligência acionável.